

A decorative graphic on the left side of the slide consisting of two overlapping parallelograms. The front one is blue and the back one is a light green color. They are positioned diagonally, with the blue one partially covering the green one.

Image Level Anomaly Detection using Convolutional Autoencoders

What is a Convolutional Autoencoder?

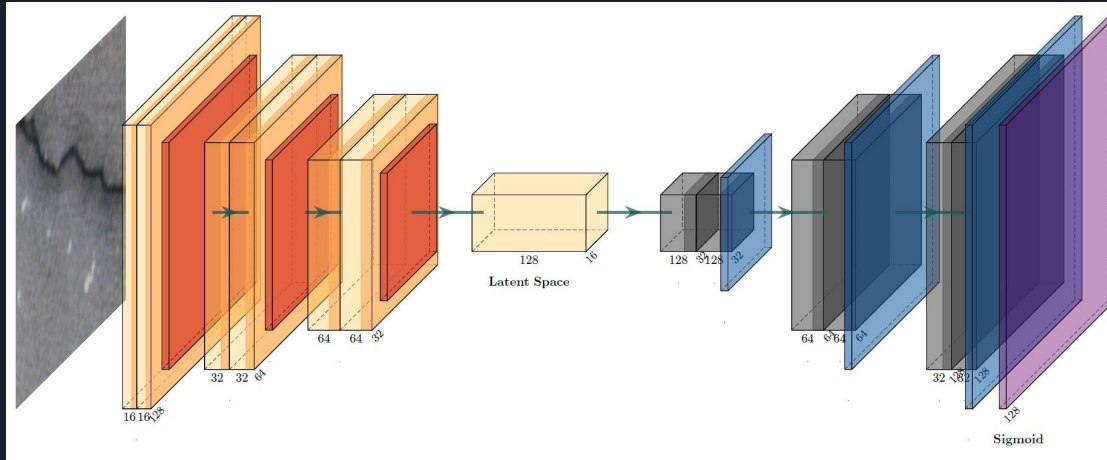
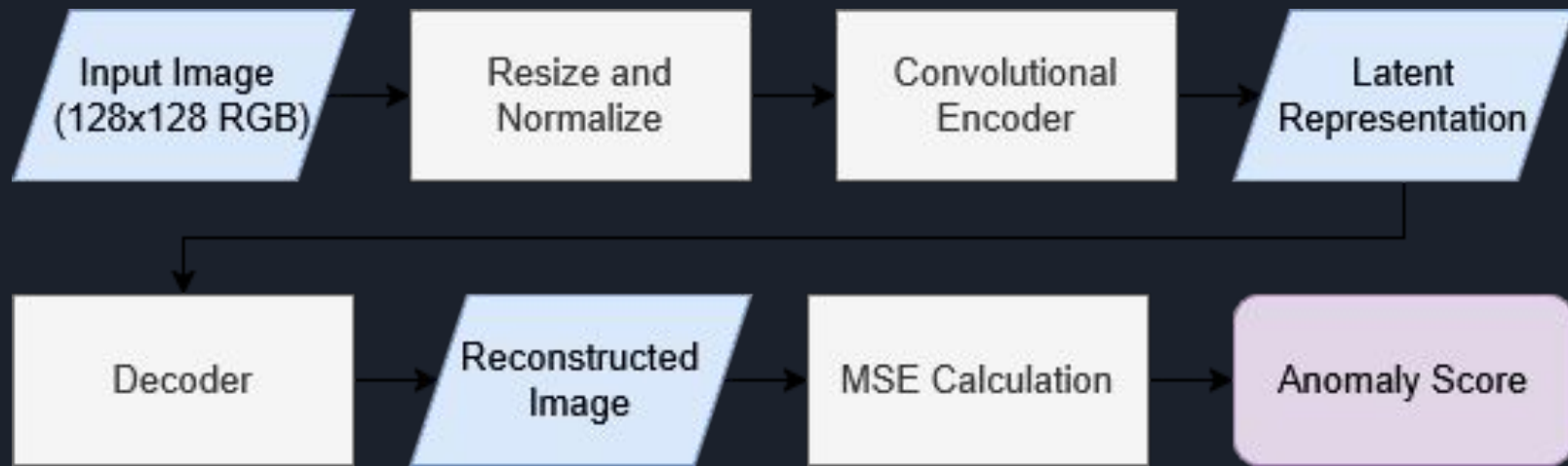


Figure 1: Convolutional Autoencoder (CAE) architecture. Source: [1]

Model Overview



Resizing and Normalizing the Input Image

Target Size of 128x128 pixels

Pixel values are scaled to $[0, 1]$

Transpose (Channels, Height, Width) for PyTorch (3, 128, 128)

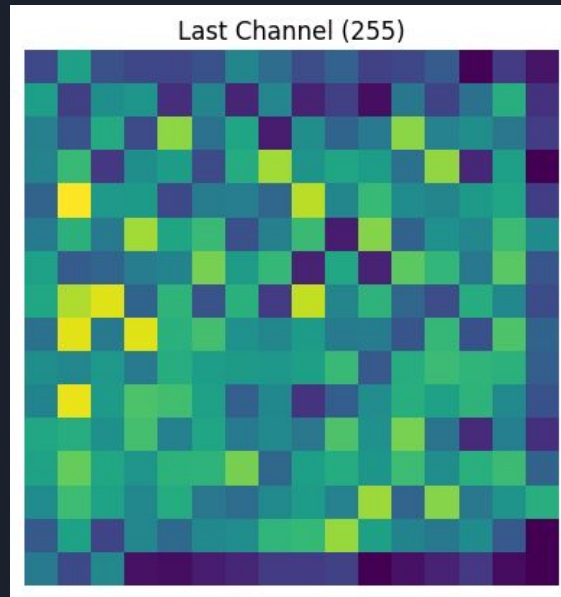
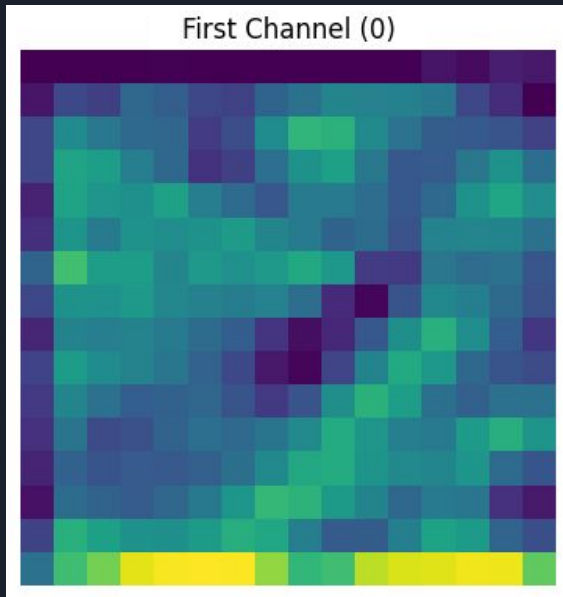


Convolutional Encoder & Latent Representation

Three convolutional layers with 3x3 kernel (padding = 1, stride = 2)

ReLU activations after each convolution

Encoder output is the latent representation (feature maps)



Reconstructed Image

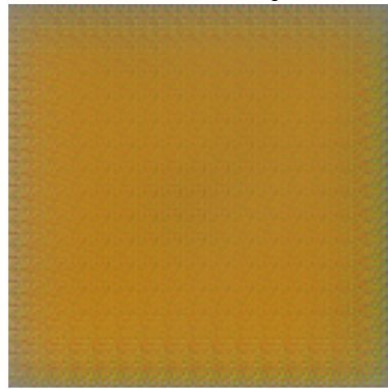
(Decoder output)

Original Input Image



50 EPOCH

Reconstructed Image



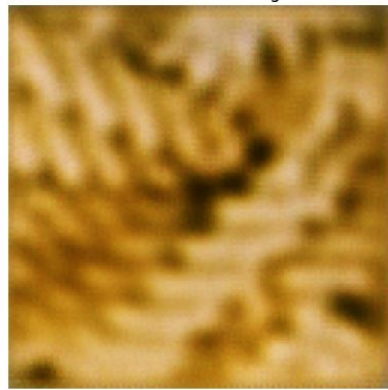
200 EPOCH

Reconstructed Image



500 EPOCH

Reconstructed Image



10000 EPOCH

Reconstructed Image





MSE Calculation & Anomaly Score

Measured the averaged squared difference between the original image and reconstructed image

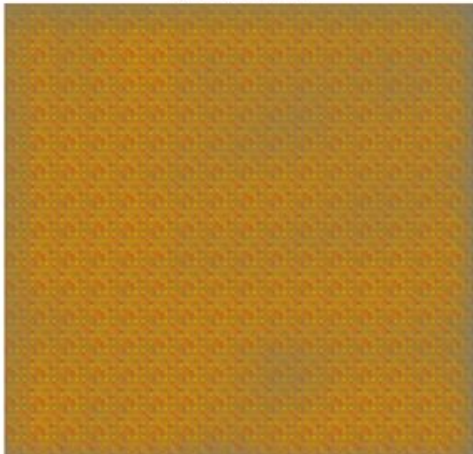
Higher MSE indicates a greater deviation from normal data, thus suggesting an anomaly

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2$$

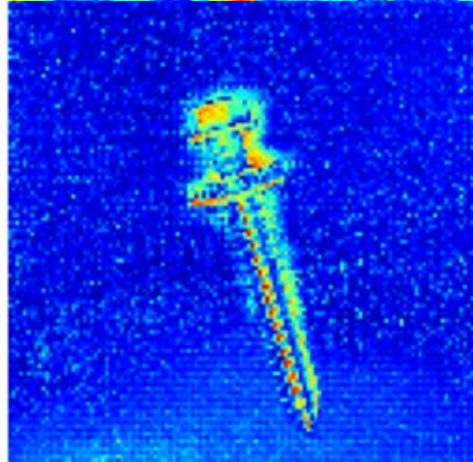
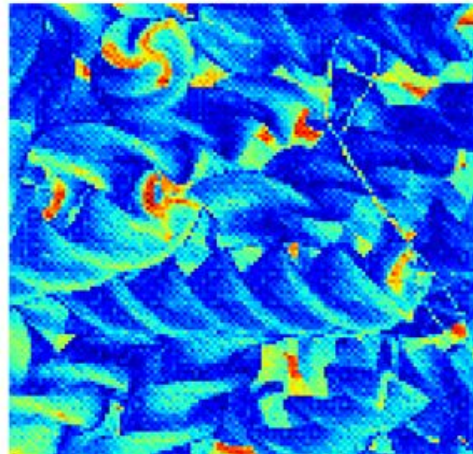
Original



Reconstructed



Error Map



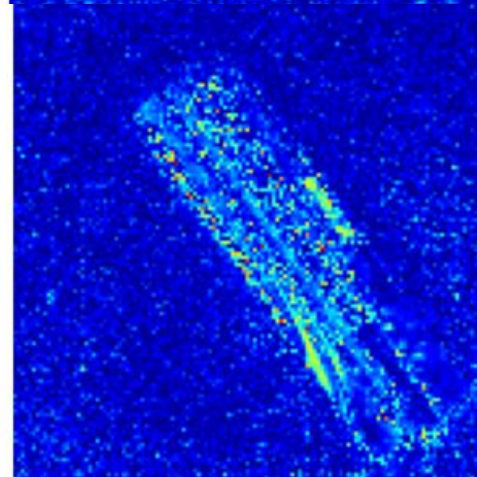
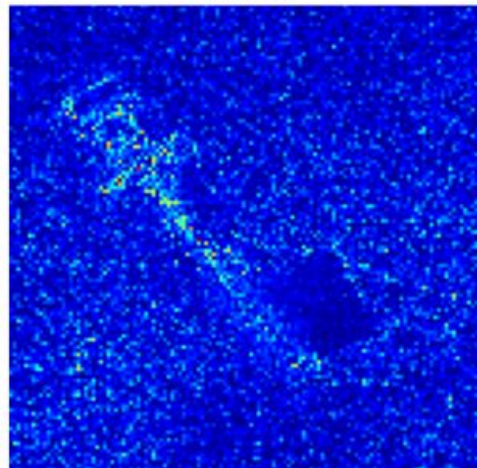
Original

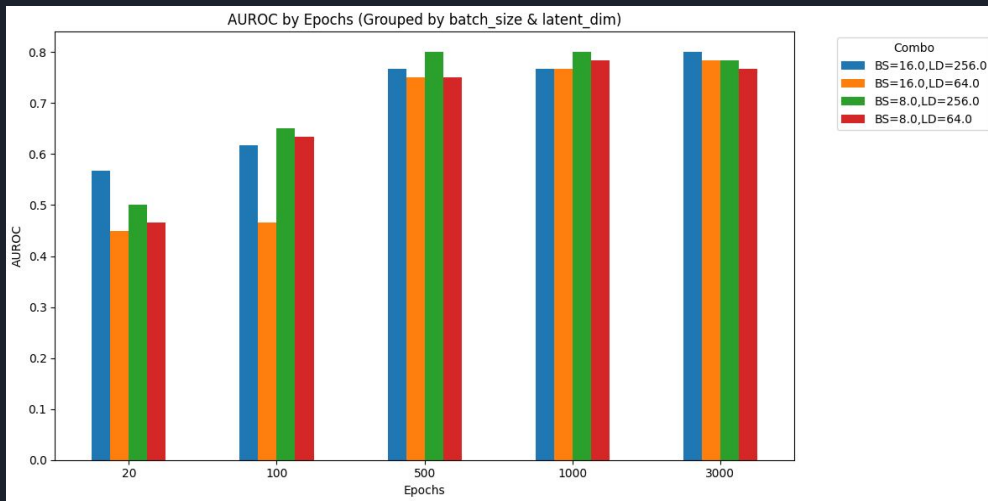


Reconstructed



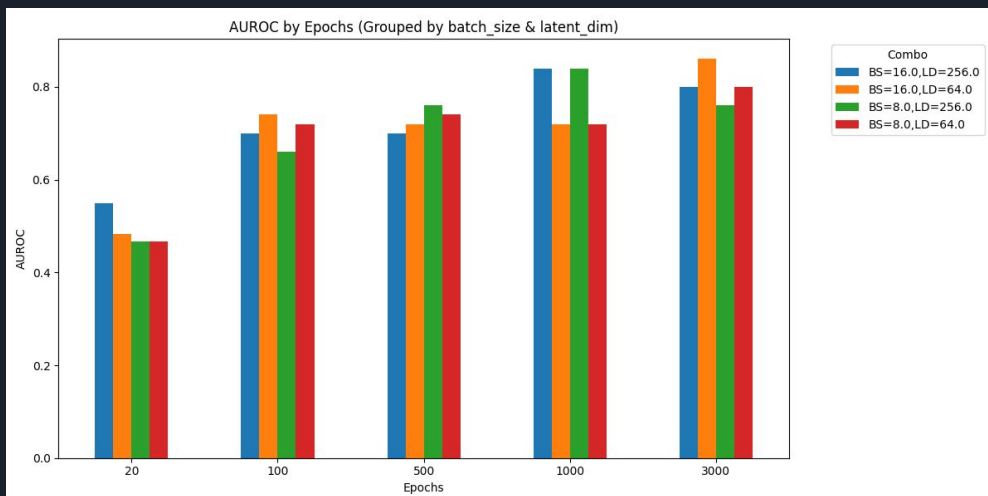
Error Map





Pasta

Top AUROC: 80%



Screws

Top AUROC: 86%



Future Work

Currently no regularization

- L1 or L2
- Denoising
- Synthetically increase dataset through transformations

Experiment with other optimizers

- Optimizing learning rate



Limitations

Hardware requirements.

- Iterations into the thousands, the speed of hardware seemed to be a significant factor

More training steps does not always mean better



References

- [1] "Convolutional Autoencoder (CAE) Architecture," *Artificial Intelligence for EM Problem Set - FIU*, [Online]. Available: <https://arc.fiu.edu/projects/artificial-intelligence-for-em-problem-set-dd/attachment/convolutional-autoencoder-cae-architecture/>. [Accessed: 30-Mar-2025].
- [2] F. Chollet, "Building Autoencoders in Keras," Keras.io, 2016. [Online]. Available: <https://blog.keras.io/building-autoencoders-in-keras.html>. [Accessed: 30-Mar-2025].
- [3] A. A. Nelay and M. Turgeon, "A Comprehensive Study of Auto-Encoders for Anomaly Detection: Efficiency and Trade-Offs," *Machine Learning with Applications*, vol. 100572, Jul. 2024. [Online]. Available: <https://doi.org/10.1016/j.mlwa.2024.100572>.
- [4] "Implementing an Autoencoder in PyTorch," *GeeksforGeeks*, Jul. 18, 2021. [Online]. Available: <https://www.geeksforgeeks.org/implementing-an-autoencoder-in-pytorch/>. [Accessed: 30-Mar-2025].
- [5] "Torch.nn — PyTorch 2.1 Documentation," *Pytorch.org*. [Online]. Available: <https://pytorch.org/docs/stable/nn.html#convolution-layers>. [Accessed: 30-Mar-2025].
- [6] A. Sharma, "Implementing a Convolutional Autoencoder with PyTorch - PyImageSearch," *PyImageSearch*, Jul. 17, 2023. [Online]. Available: <https://pyimagesearch.com/2023/07/17/implementing-a-convolutional-autoencoder-with-pytorch/>. [Accessed: 30-Mar-2025].
- [7] S.-W. Lee, H. Kim, J. Cho, and Y. H. Kim, "Towards Secure Intrusion Detection Systems Using Deep Learning Techniques: Comprehensive Analysis and Review," *Journal of Network and Computer Applications*, vol. 187, p. 103111, Aug. 2021. [Online]. Available: <https://doi.org/10.1016/j.jnca.2021.103111>. [Accessed: 30-Mar-2025].
- [8] "ML | Auto-Encoders," *GeeksforGeeks*, Jun. 21, 2019. [Online]. Available: <https://www.geeksforgeeks.org/auto-encoders/>. [Accessed: 30-Mar-2025].